

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 2
По дисциплине «Обработка изображений в интеллектуальных системах»
Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:
Студент 4 курса
Группы ИИ-26
Згера Е. А.
Проверил:
Андренко К.В.

Вариант 5

Цель: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС.

Задачи лабораторной работы

Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;

2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;

3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;

4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1).

Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;

5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

5	STL-10 (размеченная часть)	SGD	DenseNet121
---	----------------------------	-----	-------------

Код программы

```
import random
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as transforms
from torchvision import models

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
batch_size = 64
lr = 0.01
momentum = 0.9
num_classes = 10

EPOCHS_PRETRAINED = 10
EPOCHS_CUSTOM = 25
TRAIN_CUSTOM = True
FREEZE_BACKBONE = False
PRETRAINED_IMG_SIZE = 224
CUSTOM_IMG_SIZE = 96

stl10_classes = [
    "airplane", "bird", "car", "cat", "deer",
    "dog", "horse", "monkey", "ship", "truck"
]

CUSTOM_MEAN = [0.4469, 0.4399, 0.4066]
CUSTOM_STD = [0.2603, 0.2566, 0.2713]
IMNET_MEAN = [0.485, 0.456, 0.406]
IMNET_STD = [0.229, 0.224, 0.225]

custom_train_tf = transforms.Compose([
    transforms.RandomCrop(96, padding=4),
```

```

        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(mean=CUSTOM_MEAN, std=CUSTOM_STD)
    ])
custom_test_tf = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=CUSTOM_MEAN, std=CUSTOM_STD)
])

pre_train_tf = transforms.Compose([
    transforms.Resize(PRETRAINED_IMG_SIZE),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMNET_MEAN, std=IMNET_STD)
])
pre_test_tf = transforms.Compose([
    transforms.Resize(PRETRAINED_IMG_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMNET_MEAN, std=IMNET_STD)
])

def make_loaders(train_tf, test_tf):
    train_ds = torchvision.datasets.STL10(root="./data", split="train",
                                          download=True, transform=train_tf)
    test_ds = torchvision.datasets.STL10(root="./data", split="test",
                                         download=True, transform=test_tf)
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True, num_workers=0)
    test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False, num_workers=0)
    return train_ds, test_ds, train_loader, test_loader

custom_train_ds, custom_test_ds, custom_train_loader, custom_test_loader = \
    make_loaders(custom_train_tf, custom_test_tf)
pre_train_ds, pre_test_ds, pre_train_loader, pre_test_loader = \
    make_loaders(pre_train_tf, pre_test_tf)

print(f"Train: {len(pre_train_ds)}, Test: {len(pre_test_ds)}, device: {device}")

class SimpleSTL10CNN(nn.Module):
    def __init__(self):
        super(SimpleSTL10CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool1 = nn.MaxPool2d(2, 2)

        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.conv4 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
        self.pool2 = nn.MaxPool2d(2, 2)

        self.conv5 = nn.Conv2d(256, 256, kernel_size=3, padding=1)
        self.pool3 = nn.MaxPool2d(2, 2)

        self.fc1 = nn.Linear(256 * 12 * 12, 1024)
        self.fc2 = nn.Linear(1024, num_classes)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = self.pool1(x)
        x = F.relu(self.conv3(x))
        x = F.relu(self.conv4(x))
        x = self.pool2(x)
        x = F.relu(self.conv5(x))
        x = self.pool3(x)
        x = x.view(x.size(0), -1)
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)
        return x

def build_pretrained_model():
    model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

    if FREEZE_BACKBONE:

```

```

        for p in model.parameters():
            p.requires_grad = False

    in_features = model.fc.in_features
    model.fc = nn.Linear(in_features, num_classes)
    return model

def evaluate(model, loader, criterion):
    model.eval()
    total, correct, loss_sum = 0, 0, 0.0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss_sum += loss.item() * images.size(0)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    return loss_sum / total, 100 * correct / total

def train_model(model, train_loader, test_loader, num_epochs, name):
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(filter(lambda p: p.requires_grad, model.parameters()),
                           lr=lr, momentum=momentum)

    history = {"train_loss": [], "test_loss": [], "test_acc": []}
    n_train = len(train_loader.dataset)

    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * images.size(0)

        train_loss = running_loss / n_train
        test_loss, test_acc = evaluate(model, test_loader, criterion)
        history["train_loss"].append(train_loss)
        history["test_loss"].append(test_loss)
        history["test_acc"].append(test_acc)

        print(f"[{name}] Epoch {epoch + 1}/{num_epochs} | "
              f"Train Loss: {train_loss:.4f} | "
              f"Test Loss: {test_loss:.4f} | "
              f"Test Acc: {test_acc:.2f}%")
    return history

pretrained_model = build_pretrained_model().to(device)
n_trainable = sum(p.numel() for p in pretrained_model.parameters() if p.requires_grad)
print(f"ResNet18: обучаемых параметров = {n_trainable:,}")

hist_pre = train_model(pretrained_model, pre_train_loader, pre_test_loader,
                       EPOCHS_PRETRAINED, "ResNet18")
torch.save(pretrained_model.state_dict(), "resnet18_stl10.pth")

custom_model = SimpleSTL10CNN().to(device)
if TRAIN_CUSTOM:
    hist_custom = train_model(custom_model, custom_train_loader, custom_test_loader,
                              EPOCHS_CUSTOM, "CustomCNN")
    torch.save(custom_model.state_dict(), "custom_cnn.pth")
else:
    custom_model.load_state_dict(torch.load("custom_cnn.pth", map_location=device))
    l, a = evaluate(custom_model, custom_test_loader, nn.CrossEntropyLoss())
    hist_custom = {"train_loss": [], "test_loss": [l], "test_acc": [a]}

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

axes[0].plot(hist_pre["train_loss"], label="Train Loss")

```

```

axes[0].plot(hist_pre["test_loss"], label="Test Loss")
axes[0].set_title("ResNet18 (предобученная): Loss")
axes[0].set_xlabel("Эпоха")
axes[0].legend()

axes[1].plot(hist_pre["test_acc"], label="ResNet18")
if hist_custom["train_loss"]:
    axes[1].plot(hist_custom["test_acc"], label="Custom CNN (ЛР1)")
axes[1].set_title("Test Accuracy, %")
axes[1].set_xlabel("Эпоха")
axes[1].legend()

if hist_custom["train_loss"]:
    axes[2].plot(hist_pre["test_loss"], label="ResNet18")
    axes[2].plot(hist_custom["test_loss"], label="Custom CNN (ЛР1)")
    axes[2].set_title("Test Loss: сравнение")
    axes[2].set_xlabel("Эпоха")
    axes[2].legend()

plt.tight_layout()
plt.savefig("lab2_curves.png", dpi=150)
plt.show()

print("\n=== Итоги на тестовой выборке ===")
print(f"ResNet18 (pretrained): acc = {hist_pre['test_acc'][-1]:.2f}%, "
      f"loss = {hist_pre['test_loss'][-1]:.4f}, "
      f"лучшая acc = {max(hist_pre['test_acc']):.2f}%")
print(f"Custom CNN (ЛР1): acc = {hist_custom['test_acc'][-1]:.2f}%, "
      f"loss = {hist_custom['test_loss'][-1]:.4f}, "
      f"лучшая acc = {max(hist_custom['test_acc']):.2f}%")

pre_preprocess = transforms.Compose([
    transforms.Resize((PRETRAINED_IMG_SIZE, PRETRAINED_IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=IMNET_MEAN, std=IMNET_STD)
])
custom_preprocess = transforms.Compose([
    transforms.Resize((CUSTOM_IMG_SIZE, CUSTOM_IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=CUSTOM_MEAN, std=CUSTOM_STD)
])

def get_probs(model, preprocess, img):
    model.eval()
    x = preprocess(img).unsqueeze(0).to(device)
    with torch.no_grad():
        return F.softmax(model(x), dim=1).cpu().numpy().flatten()

def compare_on_custom_image(path_or_pil):
    img = Image.open(path_or_pil).convert("RGB") if isinstance(path_or_pil, str) \
    else path_or_pil.convert("RGB")

    probs_pre = get_probs(pretrained_model, pre_preprocess, img)
    probs_cus = get_probs(custom_model, custom_preprocess, img)

    fig, axes = plt.subplots(1, 3, figsize=(16, 4),
                             gridspec_kw={'width_ratios': [1, 1.4, 1.4]})
    axes[0].imshow(img)
    axes[0].axis('off')
    axes[0].set_title("Исходное изображение")

    for ax, probs, title in [(axes[1], probs_pre, "ResNet18 (предобученная)",
                              (axes[2], probs_cus, "Кастомная CNN (ЛР1)"))]:
        pred = int(np.argmax(probs))
        colors = ['red' if i == pred else 'steelblue' for i in range(num_classes)]
        ax.barh(stl10_classes, probs * 100, color=colors)
        ax.set_xlim(0, 100)
        ax.set_xlabel("Вероятность, %")
        ax.invert_yaxis()
        ax.set_title(f"{title}\nПредсказание: {stl10_classes[pred]} "
                    f"({probs[pred] * 100:.1f}%)")

    plt.tight_layout()
    plt.show()

```

```

return probs_pre, probs_cus

raw_test = torchvision.datasets.STL10(root="./data", split="test", download=True)
for idx in random.sample(range(len(raw_test)), 2):
    pil_img, lbl = raw_test[idx]
    print(f"Истинный класс: {stl10_classes[lbl]}")
    compare_on_custom_image(pil_img)

compare_on_custom_image(r"1.jpg")

```

Результат работы программы:

```

Train: 5000, Test: 8000, device: cuda
ResNet18: обучаемых параметров = 11,181,642
[ResNet18] Epoch 1/10 | Train Loss: 0.4575 | Test Loss: 0.2705 | Test Acc: 90.99%
[ResNet18] Epoch 2/10 | Train Loss: 0.1821 | Test Loss: 0.2520 | Test Acc: 92.34%
[ResNet18] Epoch 3/10 | Train Loss: 0.0867 | Test Loss: 0.2034 | Test Acc: 93.22%
[ResNet18] Epoch 4/10 | Train Loss: 0.0707 | Test Loss: 0.2132 | Test Acc: 93.40%
[ResNet18] Epoch 5/10 | Train Loss: 0.0444 | Test Loss: 0.1890 | Test Acc: 94.11%
[ResNet18] Epoch 6/10 | Train Loss: 0.0163 | Test Loss: 0.2101 | Test Acc: 93.70%
[ResNet18] Epoch 7/10 | Train Loss: 0.0163 | Test Loss: 0.2016 | Test Acc: 94.09%
[ResNet18] Epoch 8/10 | Train Loss: 0.0085 | Test Loss: 0.1720 | Test Acc: 94.56%
[ResNet18] Epoch 9/10 | Train Loss: 0.0097 | Test Loss: 0.1869 | Test Acc: 94.42%
[ResNet18] Epoch 10/10 | Train Loss: 0.0047 | Test Loss: 0.1707 | Test Acc: 95.00%
[CustomCNN] Epoch 1/25 | Train Loss: 2.2705 | Test Loss: 2.1836 | Test Acc: 18.98%
[CustomCNN] Epoch 2/25 | Train Loss: 1.9984 | Test Loss: 1.7710 | Test Acc: 33.64%
[CustomCNN] Epoch 3/25 | Train Loss: 1.7425 | Test Loss: 1.5639 | Test Acc: 41.24%
[CustomCNN] Epoch 4/25 | Train Loss: 1.6202 | Test Loss: 1.5810 | Test Acc: 41.02%
[CustomCNN] Epoch 5/25 | Train Loss: 1.6005 | Test Loss: 1.5060 | Test Acc: 43.62%
[CustomCNN] Epoch 6/25 | Train Loss: 1.4485 | Test Loss: 1.3354 | Test Acc: 49.95%
[CustomCNN] Epoch 7/25 | Train Loss: 1.3593 | Test Loss: 1.3704 | Test Acc: 49.66%
[CustomCNN] Epoch 8/25 | Train Loss: 1.3033 | Test Loss: 1.4395 | Test Acc: 48.21%
[CustomCNN] Epoch 9/25 | Train Loss: 1.2474 | Test Loss: 1.1903 | Test Acc: 55.29%
[CustomCNN] Epoch 10/25 | Train Loss: 1.1572 | Test Loss: 1.1505 | Test Acc: 57.76%
[CustomCNN] Epoch 11/25 | Train Loss: 1.1127 | Test Loss: 1.2309 | Test Acc: 55.90%
[CustomCNN] Epoch 12/25 | Train Loss: 1.0847 | Test Loss: 1.1309 | Test Acc: 58.45%
[CustomCNN] Epoch 13/25 | Train Loss: 1.0411 | Test Loss: 1.2769 | Test Acc: 55.33%
[CustomCNN] Epoch 14/25 | Train Loss: 0.9962 | Test Loss: 1.0380 | Test Acc: 62.08%
[CustomCNN] Epoch 15/25 | Train Loss: 0.8847 | Test Loss: 1.0454 | Test Acc: 62.30%
[CustomCNN] Epoch 16/25 | Train Loss: 0.8697 | Test Loss: 0.9986 | Test Acc: 64.86%
[CustomCNN] Epoch 17/25 | Train Loss: 0.8168 | Test Loss: 1.0157 | Test Acc: 63.66%
[CustomCNN] Epoch 18/25 | Train Loss: 0.7636 | Test Loss: 0.9692 | Test Acc: 66.04%
[CustomCNN] Epoch 19/25 | Train Loss: 0.7088 | Test Loss: 1.0569 | Test Acc: 63.56%
[CustomCNN] Epoch 20/25 | Train Loss: 0.7384 | Test Loss: 0.9698 | Test Acc: 66.47%
[CustomCNN] Epoch 21/25 | Train Loss: 0.6482 | Test Loss: 0.9628 | Test Acc: 67.91%
[CustomCNN] Epoch 22/25 | Train Loss: 0.5894 | Test Loss: 1.0086 | Test Acc: 66.97%
[CustomCNN] Epoch 23/25 | Train Loss: 0.5573 | Test Loss: 0.9500 | Test Acc: 67.91%
[CustomCNN] Epoch 24/25 | Train Loss: 0.5244 | Test Loss: 0.9839 | Test Acc: 67.45%
[CustomCNN] Epoch 25/25 | Train Loss: 0.4474 | Test Loss: 1.0480 | Test Acc: 66.72%

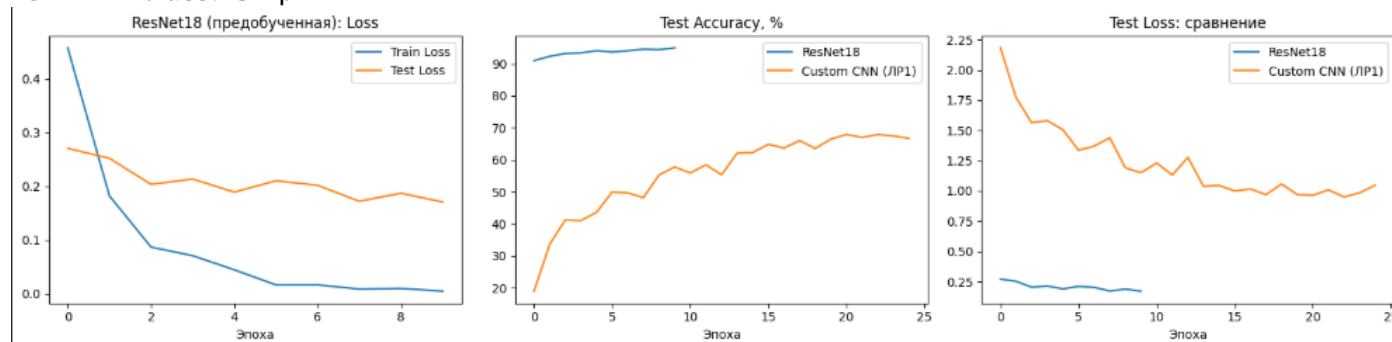
```

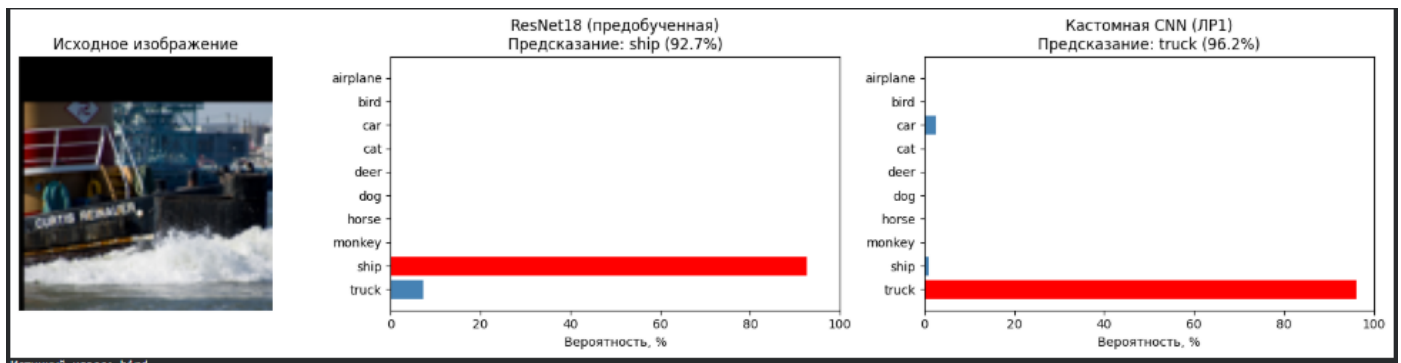
=== Итоги на тестовой выборке ===

ResNet18 (pretrained): acc = 95.00%, loss = 0.1707, лучшая acc = 95.00%

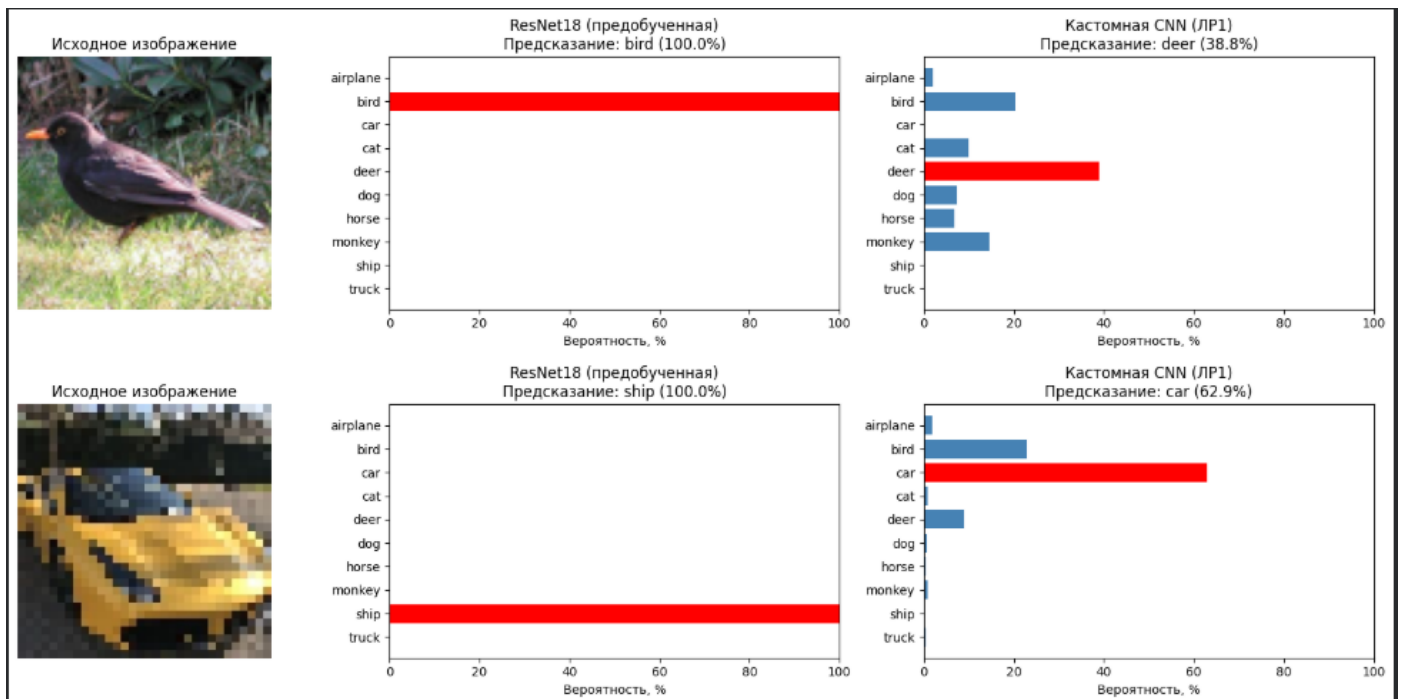
Custom CNN (ЛР1): acc = 66.72%, loss = 1.0480, лучшая acc = 67.91%

Истинный класс: ship





Истинный класс: bird



Показатель	Кастомная СНС (ЛР %1)	ResNet18 (предобученная)
Эпох обучения	25	10
Test Accuracy (финальная)	66.72%	95.00%
Лучшая Test Accuracy	67.91%	95.00%
Test Loss (финальный)	1.048	0.171
Train Loss (финальный)	0.447	0.005
Точность после 1-й эпохи	18.98%	90.99%

Предобученная сеть превосходит кастомную примерно на 28 п.п. по точности, а ошибка на тесте у неё меньше в 6 раз. Она достигла 91% уже после первой эпохи, тогда как кастомной сети для 67% потребовалось 25 эпох. Причина в том, что ResNet18 использует признаки (границы, текстуры, формы объектов), уже выученные на миллионах изображений ImageNet, а кастомная сеть вынуждена изучать их с нуля на 5000 обучающих примерах.

У кастомной сети виден заметный разрыв между ошибкой на обучении (0.45) и на тесте (1.05), то есть переобучение. Кривая тестовой ошибки практически перестала снижаться после 18-й эпохи, точность колебалась в диапазоне 66–68%. У ResNet18 разрыв по loss также есть (0.005 против 0.17), но он не мешает высокой точности, так как предобученные признаки хорошо обобщаются.

Сравнение с state-of-the-art

Лучшие опубликованные результаты на STL-10 составляют около 99.7% точности (крупные предобученные модели вроде ViT-L/16). Полученные результаты соотносятся с ними так:

- ResNet18 отстаёт от SOTA примерно на 4.7 п.п. (5% ошибок против ~0.3%). Разрыв объясняется меньшим размером модели (~11 млн параметров), небольшой длительностью дообучения, отсутствием сложных аугментаций и расписания скорости обучения, а также меньшим объёмом предобучения по сравнению с лучшими моделями.
- Кастомная СНС отстаёт примерно на 33 п.п. Для простой архитектуры из базовых слоёв (свёрточные, подвыборочные, полносвязные, ReLU), обученной с нуля только на 5000 размеченных изображениях, результат 65–70% является типичным.

Использование предобученной архитектуры (transfer learning) существенно повышает качество классификации и ускоряет сходимость на небольшой выборке: точность выросла с 66.7% до 95.0% при снижении числа эпох с 25 до 10. Кастомная модель подтверждает, что простая сеть без предобучения ограничена малым объёмом данных.

Вывод: Прodelав данную лабораторную работу я научился конструировать нейросетевые классификаторы и выполнять из обучение на известных выборках компьютерного зрения.